DJANGO TEXTBOOK SERIES [FILE 03] · VISUAL PRINCIPLES TEXTBOOK (LECTURES 1–11)

[3번째 파일] 한눈에 이해하는 Django 핵심 원리 시각 교과서

명령어 단순 암기보다 '왜(Why) 이렇게 작동하는가'가 우선인 11대 핵심 메커니즘 + 현업 시니어 개발자가 짚어주는 초보자 단골 함정 & 실전 디버깅 비법 완벽 총정리

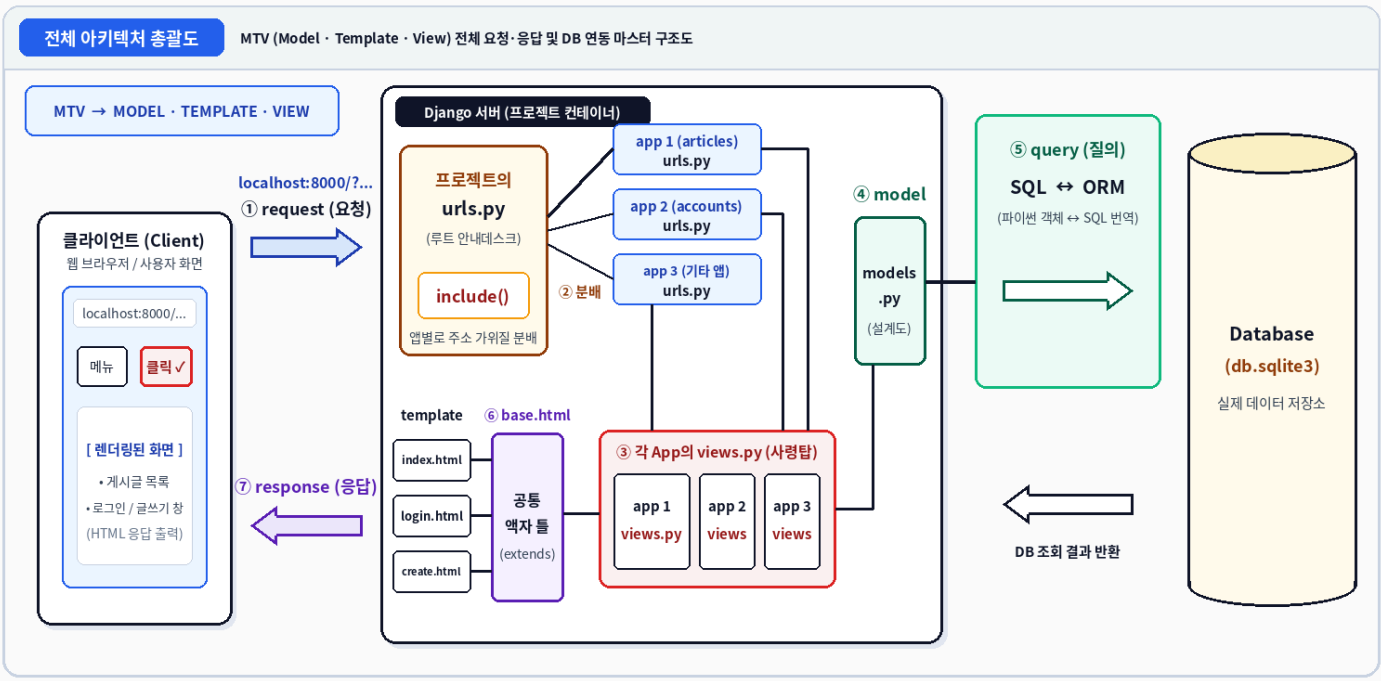
웹 프레임워크인 Django를 처음 배울 때 가장 흔히 겪는 어려움은 **"명령어와 코드는 외워서 쳤는데, 왜 이 파일에 코드를 적어야 하고 왜 조금만 응용하면 에러가 나는지 모르겠다"**는 점입니다. 본 교재는 총 11권의 Django 정규 과정(Intro, Template & URLs, Model, UI/UX & 반응형 웹, ORM, ORM with View, Django Form, Authentication 1·2·3) 전체에서 반드시 작동 원리와 인과관계를 이해해야 하는 핵심 메커니즘을 시각 도해(`fig0` ~ `fig10`)로 풀어내고, 초보 개발자들이 실전에서 가장 많이 오해하거나 실수하는 기술적 함정(Gotchas)까지 시니어 개발자 시선에서 정밀하게 교정·보완한 개념 기본서입니다.

단원 구분	교재 범위 및 주제	암기 대신 반드시 이해해야 할 핵심 질문 (Why?)	교과서 비유 & 베테랑 실전 교정 포인트
제 1 단원	교재 1권: 전체 구조와 가상환경	클라이언트 요청이 어떤 파일들을 거쳐 DB까지 갔다 오며, 왜 `venv`를 만들까?	[총괄 도해] MTV 풀스택 지도 / 원룸 vs 독립된 방과 레시피
제 2 단원	교재 1권: MTV 디자인 패턴	왜 코드를 Model·Template·View로 쪼개고, 템플릿 폴더 안에 앱 이름을 또 만들까?	백화점 분업 / `APP_DIRS` 원리와 공통 `base.html`의 `DIRS` 등록법
제 3 단원	교재 2권: 템플릿 상속과 URL 분배	왜 `base.html`을 상속받고, `include()`와 `app_name:name`으로 주소를 관리할까?	액자 틀 교체 / `extends` 제1줄 규칙 & `urls.py` 후행 슬래시(`/`) 필수 이유
제 4 단원	교재 3권: Model과 마이그레이션	왜 `models.Model`을 상속받고, DB 반영을 2단계(`makemigrations`→`migrate`)로 나눌까?	거인의 어깨 / 문자열 필드 `null=True` 금지 이유 & 기본값(`default`) 질문 대처법
제 5 단원	교재 4권: UI·UX와 반응형 웹	왜 스마트폰에서 `viewport` 태그가 필수이며, GitHub Pages에는 Django를 못 올릴까?	스마트폰 교정 안경 / `min-width`(오름차순) vs `max-width`(내림차순) 순서
제 6 단원	교재 5권: ORM과 QuerySet API	왜 `all()`·`filter()`와 `get()`의 반환값이 다르고, `save()`는 생성·수정을 어떻게 구분할까?	바구니 vs 알맹이 / QuerySet 지연 평가(Lazy Evaluation) & `get_object_or_404`
제 7 단원	교재 6권: HTTP 방식·CSRF·Redirect	왜 `POST`에 `csrf_token`이 없으면 `403` 에러가 나고, 저장 후 `redirect()`를 쓸까?	엽서 vs 봉투 / `GET` 삭제 시 보안 참사 & HTTP 상태 코드(302·403·404·500)
제 8 단원	교재 7권: Form과 단일 View 통합	왜 두 View 함수를 합칠 때 `return render()`가 맨 아래(`else` 바깥)에 위치할까?	보안 검문소 / `ArticleForm(article)` 위치 인자 실수 & `save(commit=False)`
제 9 단원	교재 8·9권: 쿠키·세션과 인증 체계	왜 첫 `migrate` 전에 User 모델을 바꾸고, 탈퇴 시 `delete()`를 `logout()`보다 먼저 할까?	금고 열쇠(`sessionid`) / 인증 4대 폼 제1인자 구분 & `update_session_auth_hash`
제 10 단원	교재 10권: 비번 암호화·수정·초기화	왜 비밀번호를 해시+솔트+키 스트레칭(`PBKDF2`)으로 저장하고, `auth.urls`는 아랫줄에 둘까?	과일 믹서기 주스 / 레인보우 테이블↔솔트, 무차별 대입↔키 스트레칭 & 초기화
제 11 단원	교재 11권: PJT-04 Model·Form·Auth 심화	왜 `choices`에 `PositiveIntegerField`를 더하고, `cleaned_data`·`Meta.widgets`·`?next=` 검문소를 쓸까?	자판기 코드 / 공항 검색대 / 벽지 인테리어 vs 가구 교체 / 포스트잇 알림 & 택시 검문소

제 1 단원 : Django 전체 아키텍처 지도와 클라이언트·서버 및 가상환경(venv) 원리 (교재 1권)

0. [총괄 마스터 지도] 숲부터 보기: 클릭 한 번이 Django 내부 파일들과 DB를 거쳐 돌아오는 7단계 여정

개별 문법을 배우기 전에 반드시 머릿속에 사진을 찍듯 넣어두어야 할 **'Django MTV 풀스택 전체 작동 지도'**입니다. 사용자가 브라우저 화면의 버튼을 클릭(`request`)했을 때, 프로젝트 안의 파일들이 어떻게 바통을 주고받으며 데이터베이스(DB)에서 자료를 꺼내와 완성된 화면(`response`)을 돌려주는지 아래 그림의 ①번부터 ⑦번 순서를 따라가며 눈으로 확인해 보세요.

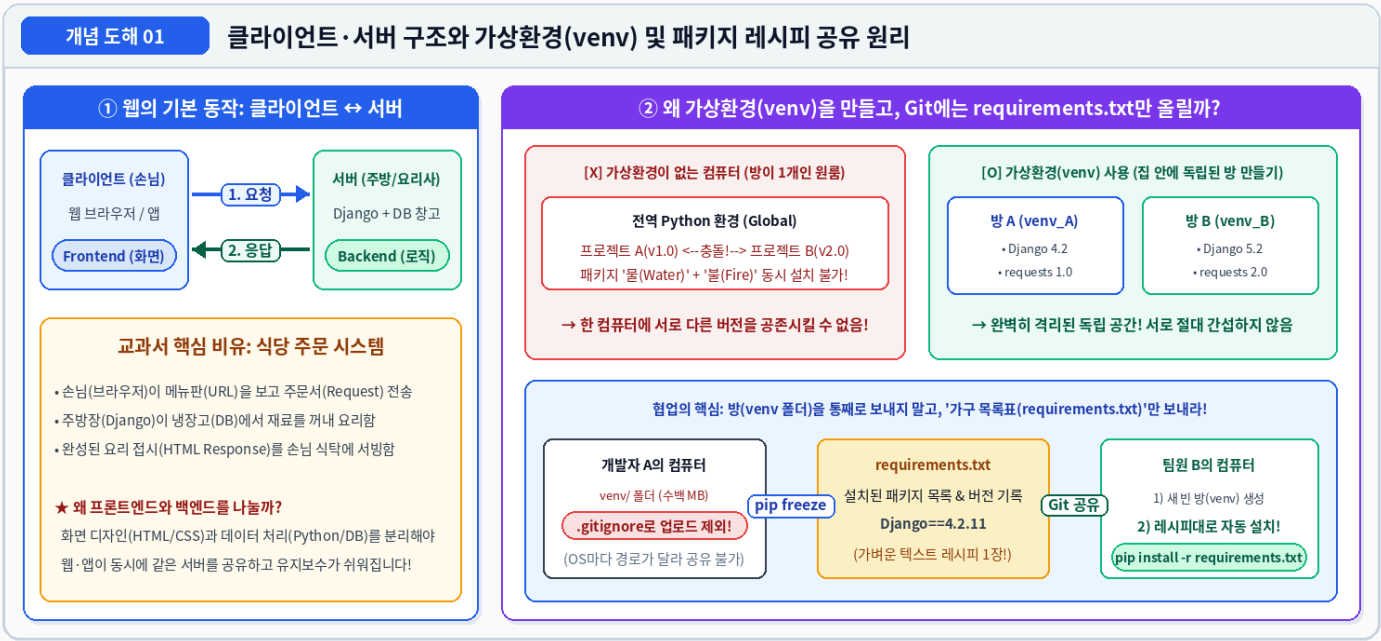


[총괄 도해] 클라이언트 요청(request) → 프로젝트 urls.py(include) → 각 App urls.py → views.py → models.py(ORM/SQL) ↔ DB → base.html + 하위 템플릿 → 최종 응답(response)

- ① **요청(`request`) 진입**: 사용자가 브라우저에서 버튼을 클릭하면 `localhost:8000/...` 주소로 요청이 날아와 가장 먼저 프로젝트의 `urls.py` 정문에 도착합니다.
- ② **`include()` 가위질 분배**: 프로젝트의 `urls.py`는 주소 앞부분을 보고 각 담당 앱(`app 1`, `app 2`, `app 3`)의 `urls.py`로 길을 나누어 안내합니다.
- ③ **중앙 사령탑(각 App의 `views.py`) 출동**: 각 앱의 `urls.py`는 연결된 `views.py` 함수를 호출합니다.
- ④·⑤ **DB 조회(`model` ↔ `ORM / SQL query` ↔ `Database`)**: 데이터가 필요하면 `views.py`가 `models.py`를 부르고, ORM이 파이썬 명령을 SQL 질의(Query)로 번역해 우측 원통형 DB(`db.sqlite3`)에서 데이터를 꺼내 옵니다.
- ⑥·⑦ **화면 합성(`base.html` + 하위 `template`) 및 응답(`response`)**: `views.py`가 가져온 데이터를 공통 액자 틀인 `base.html`과 개별 조각 템플릿(`index.html`, `login.html`, `create.html`)에 입혀 완성된 HTML을 클라이언트에게 돌려줍니다!

1. 웹의 기본 작동 원리: 클라이언트(손님)와 서버(주방)의 요청·응답 줄다리기

웹 서비스는 결코 혼자 작동하지 않습니다. 크롬 브라우저나 모바일 앱(**클라이언트, Client**)이 서버로 **요청(Request)**을 보내면, 24시간 대기 중인 컴퓨터(**서버, Server**)가 DB에서 재료를 꺼내 요리한 뒤 HTML 문서 또는 JSON 데이터로 **응답(Response)**을 돌려줍니다. 이때 사용자 눈에 보이는 화면 영역을 **프론트엔드(Frontend: HTML, CSS, JS, Vue.js)**라 하고, 뒤편에서 요청 처리와 DB를 담당하는 영역을 **백엔드(Backend: Python Django)**라고 부릅니다.



[그림 1] 클라이언트·서버 요청·응답 구조와 가상환경(venv) 독립 공간 및 레시피(requirements.txt) 공유 메커니즘

2. 왜 가상환경(venv)을 반드시 만들고, Git에는 requirements.txt만 공유할까?

파이썬을 그냥 설치하면 '전역(Global) 공간'이라는 단 하나의 거실만 생겨서, `requests 1.0`이 필요한 프로젝트 A와 `requests 2.0`이 필요한 프로젝트 B가 한 컴퓨터에서 충돌합니다. 그래서 프로젝트마다 독립된 방(`python -m venv venv`)을 따로 짓는 것입니다. 또한 `venv/` 폴더는 수백 MB로 무겁고 OS마다 내부 구조(Windows는 `Scripts/`, Mac/Linux는 `bin/`)가 달라 공유하면 에러가 나므로, `.gitignore`로 제외하고 오직 가구 목록표 레시피인 `requirements.txt`(`pip freeze > requirements.txt`)만 공유합니다.

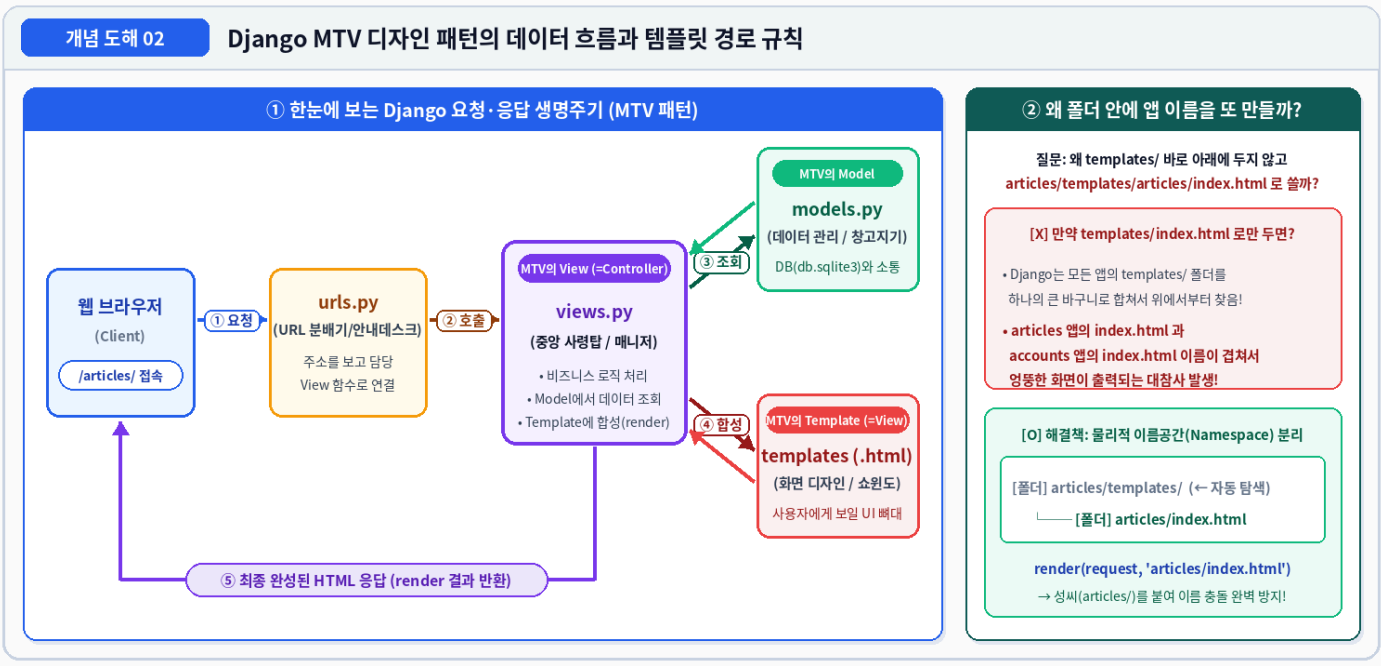
- **시니어 개발자의 실전 디버깅 코칭 — 초보자가 가상환경에서 가장 많이 겪는 3가지 함정!**
  ① **터미널별 활성화 명령어 차이**: `source venv/Scripts/activate`는 **Windows Git Bash 전용**입니다! Windows CMD/PowerShell에서는 `venv\Scripts\activate`를, Mac/Linux에서는 `source venv/bin/activate`를 쳐야 합니다.
  ② **프롬프트 `(venv)` 확인 철칙**: 반드시 터미널에 `(venv)` 표시가 떠 있는지 눈으로 확인한 뒤 `pip install django`를 실행하세요.
  ③ **`venv`가 있는 폴더는 절대 이름 변경·이동 금지!**: `venv` 내부에는 생성 당시의 '절대 경로'가 박혀 있어 폴더 이름을 바꾸거나 옮기면 가상환경이 즉시 고장 납니다. 폴더를 옮겼다면 기존 `venv`를 지우고 `requirements.txt`로 새로 만드세요!

제 2 단원 : Django MTV 디자인 패턴과 요청·응답 생명주기 (교재 1권)

1. 왜 코드를 한 파일에 안 쓰고 MTV(Model-Template-View) 3개로 쪼개 놓았을까?

초보 시절에는 파일 하나에 DB 접속 코드, 계산 로직, 화면 HTML을 전부 섞어 쓰는 것이 편해 보입니다. 하지만 서비스 규모가 커지면 글자 색깔 하나를 바꾸려다 결제 DB 코드가 망가지는 대참사가 일어납니다. 그래서 소프트웨어 공학에서는 **'관심사의 분리 (Separation of Concerns)'** 원칙에 따라 역할을 3등분한 MVC 디자인 패턴을 사용하며, Django는 이를 자기만의 철학에 맞춰 이름만 바꾼 **MTV 디자인 패턴**을 채택했습니다.

Django (MTV)	표준 MVC 대응	담당 파일 및 비유	핵심 수행 업무 및 시니어 개발자의 정밀 교정
M (Model)	Model (동일)	`models.py` (창고지기)	데이터베이스(DB)의 테이블 구조(Schema)를 정의하고 데이터를 저장·조회·수정·삭제함
T (Template)	**View** (이름 주의!)	`templates/*.html` (쇼윈도)	사용자의 눈에 보이는 화면(UI) 뼈대를 만듦. **주의: DTL은 화면 출력(Presentation)만 담당하며, DB 조작이나 복잡한 파이썬 연산은 절대 여기서 하지 않음!**
V (View)	**Controller**의 핵심	`views.py` (중앙 사령탑)	HTTP 요청(`request`)을 받아 Model에서 데이터를 가져오고 가공한 뒤, 어떤 Template에 입혀 응답(`HttpResponse`)할지 결정하는 비즈니스 로직의 중심
URLconf + 프레임워크	Controller의 라우터	`urls.py` (안내데스크)	엄밀히 말하면 Django에서는 프레임워크 자체와 `urls.py` + `views.py`가 합쳐져 MVC의 Controller 역할을 수행함



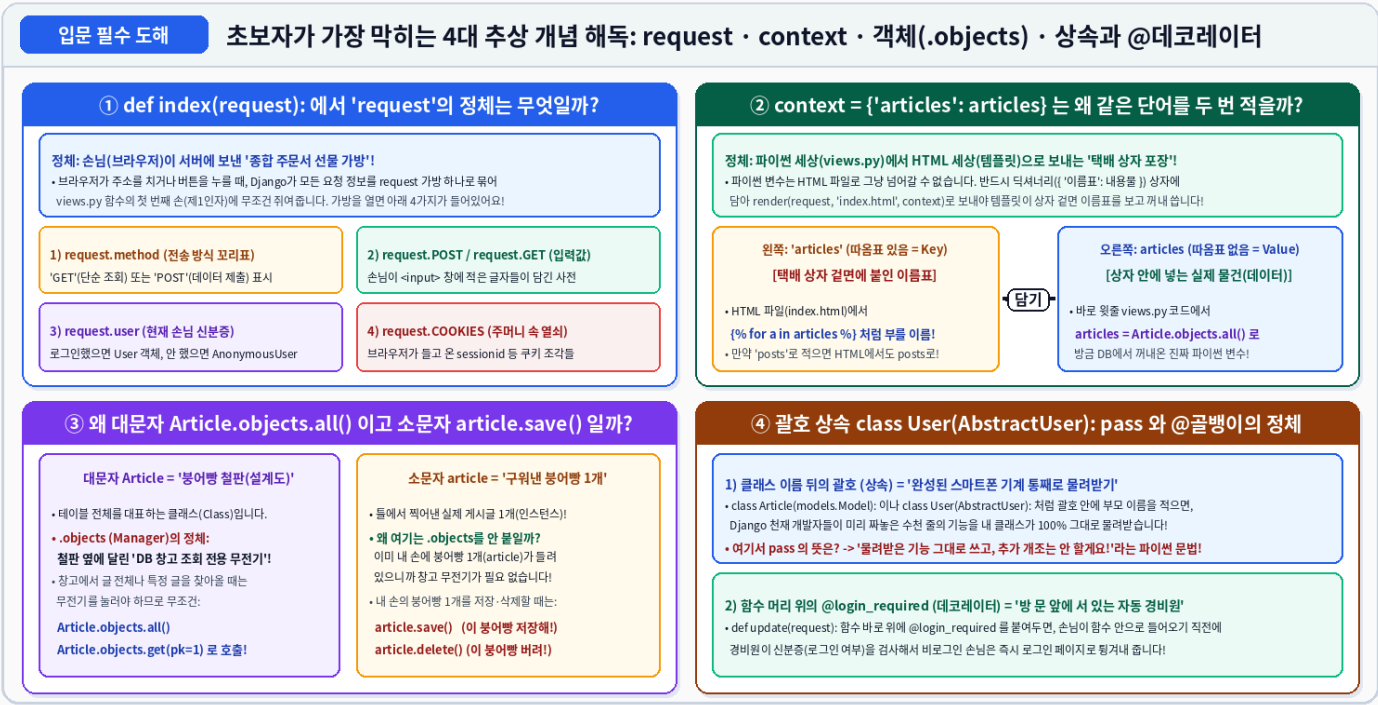
[그림 2] Django MTV 디자인 패턴의 5단계 요청·응답 생명주기와 템플릿 네임스페이스(articles/templates/articles/) 구조

2. 프로젝트 vs 앱 등록 순서와 템플릿 탐색 경로(APP_DIRS vs DIRS)의 진짜 원리

- **왜 `startapp`으로 매장을 먼저 지은 뒤 `INSTALLED_APPS`에 등록할까?** 아직 존재하지도 않는 유령 매장을 백화점 안내판(`settings.py`)에 먼저 적어두면 Django가 앱을 찾지 못해 에러가 납니다. 반드시 ① 생성(`python manage.py startapp articles`) → ② 등록(`INSTALLED_APPS`) 순서를 지켜야 합니다.
- **왜 `articles/templates/articles/index.html`처럼 폴더 안에 앱 이름을 또 만들까?** `settings.py`의 `TEMPLATES` 설정을 보면 `'APP_DIRS': True`가 켜져 있습니다. 이 옵션 때문에 Django는 모든 앱 내부의 `templates/` 폴더들을 `INSTALLED_APPS`**에 등록된 순서대로 하나의 큰 바구니처럼 합쳐서 탐색**합니다. 만약 성씨 폴더(`articles/`) 없이 `templates/index.html`로만 두면, 뒤에 등록된 `accounts` 앱의 `index.html`을 불러도 윗줄에 있는 `articles` 앱의 `index.html`이 먼저 가로채어 출력되는 버그가 발생합니다!
- **[베테랑 필수 보충] 프로젝트 전체가 공유하는 `base.html`은 왜 바로 못 찾을까?** `'APP_DIRS': True`는 오직 '등록된 앱 폴더 안의 `templates/`'만 자동으로 뒤집니다! 따라서 특정 앱에 속하지 않는 최상위 `templates/base.html`을 만들 때는 반드시 `settings.py`의 `TEMPLATES` 안 빈 리스트인 `'DIRS': [BASE_DIR / 'templates'],`에 경로를 직접 적어주어야 `TemplateDoesNotExist` 에러가 나지 않습니다.

3. [입문자 필수 해독] 코딩하다 가장 많이 막히는 4대 추상 개념의 진짜 정체 (request · context · .objects · 상속)

코드를 따라 치다 보면 *"왜 함수마다 괄호 안에 `request`를 적지?", "왜 `{'articles': articles}`처럼 똑같은 단어를 두 번 적지?", "왜 어떤 건 대문자 `Article.objects`이고 어떤 건 소문자 `article.save()`지?"*라는 의문이 반드시 생깁니다. 앞으로 계속 등장할 4가지 핵심 추상 개념의 정체를 아래 그림으로 먼저 깨치고 가세요!



[입문 필수 도해] 초보자가 가장 많이 막히는 4대 추상 개념 해독: ① request 선물 가방 · ② context 택배 상자 · ③ 대문자 클래스(.objects 무전기) vs 소문자 인스턴스(붕어빵 1개) · ④ 괄호 상속(pass)과 @데코레이터 경비원

초보자가 막히는 코드	이해하기 쉬운 한 줄 정체 (비유)	왜 이렇게 쓰며, 안에는 무엇이 들어있을까?
① `def index(request):`의 `request`	손님(브라우저)이 서버에 보낸 **'종합 주문서 선물 가방'**	브라우저가 요청을 보낼 때 Django가 모든 정보를 `request` 가방 하나로 묶어 View 함수 첫 번째 인자로 쥐여줍니다. 가방을 열면 전송 방식(`request.method`), 사용자가 입력한 글자(`request.POST`), 현재 로그인한 회원 신분증(`request.user`)이 다 들어있습니다!
② `{'articles': articles}`의 `context` 딕셔너리	파이썬에서 HTML로 보내는 **'이름표 붙인 택배 상자'**	파이썬 변수를 HTML 파일로 그냥 던질 수 없어서 상자에 포장해 보냅니다. 왼쪽 따옴표 있는 `'articles'`(Key)는 **HTML 파일에서 `{{ articles }}`로 부를 택배 상자 겉면 이름표**이고, 오른쪽 따옴표 없는 `articles`(Value)는 **방금 DB에서 꺼낸 실제 데이터 물건**입니다!
③ `Article.objects.all()` vs `article.save()`	**대문자 = '붕어빵 철판 + 창고 무전기'** **소문자 = '구워낸 붕어빵 1개'**	대문자 `Article`은 테이블 전체 설계도이고 `.objects`(Manager)는 **DB 창고 조회 전용 무전기**입니다. 그래서 목록을 찾을 땐 무전기(`Article.objects.all()`)를 누르고, 이미 내 손에 쥔 붕어빵 1개(소문자 `article`)를 저장·삭제할 땐 무전기 없이 `article.save()`, `article.delete()`를 직접 부릅니다!

초보자가 막히는 코드	이해하기 쉬운 한 줄 정체 (비유)	왜 이렇게 쓰며, 안에는 무엇이 들어있을까?
④ `(models.Model)` · `pass` & `@login_required`	**괄호 상속 = '완성된 기계 물려받기'** **@ 골뱅이 = '문 앞 자동 경비원'**	클래스 옆 괄호 안에 부모 이름을 적으면 Django가 미리 짜놓은 수천 줄의 기능을 100% 물려받으며, **`pass`는 "물려받은 기능 그대로 쓰고 추가 개조는 안 할게요!"**라는 뜻입니다. 함수 위의 `@login_required`는 함수에 들어가기 전 신분증을 검사해 비로그인 손님을 로그인 창으로 튕겨내는 경비원입니다.

제 3 단원 : 템플릿 상속(extends)과 URL 분배기(urls.py)의 원리 (교재 2권)

1. 왜 HTML마다 메뉴바를 복사하지 않고 {% extends %}와 {% block %}을 쓸까?

웹사이트의 모든 페이지에는 똑같은 상단 내비게이션 바와 Bootstrap CDN 링크가 들어갑니다. 100개의 HTML 파일에 이를 복사·붙여넣기를 했다면 메뉴 하나를 바꿀 때 100개의 파일을 다 고쳐야 합니다. 그래서 부모 템플릿(`base.html`)에 공통 액자 틀과 교체용 창문(`{% block content %}`)을 뚫어두고, 자식 템플릿이 물려받아 알맹이만 갈아 끼우는 것입니다.

- **[베테랑 디버깅 포인트 1] `{% extends 'base.html' %}`은 반드시 파일의 '가장 첫 번째 줄(1번 줄)'에 와야 합니다!** 그 위에 빈 줄이 아닌 일반 HTML 태그나 심지어 주석이라도 먼저 적혀 있으면 `TemplateSyntaxError: 'extends' must be the first tag in the template` 에러가 발생합니다.
- **[베테랑 디버깅 포인트 2] 자식 템플릿에서 `{% block %}` 바깥에 적은 코드는 화면에 아예 안 나옵니다!** 부모 액자의 지정된 창문(`block`) 안쪽에 넣은 내용만 교체되어 들어가므로, 자식 템플릿에서 `block` 바깥에 글자를 적으면 에러도 없이 유령처럼 무시됩니다.



[그림 3] 템플릿 상속(base.html 액자 틀 + block 창문)과 URL 분배기의 3대 메커니즘(include 가위질 · 변수 라우팅 · 이름표)

2. HTML <form> 태그의 Throw-Catch와 URL 분배(Dispatcher)의 3대 핵심 원리

`<form action="/catch/" method="GET">` 내부의 `<input type="text" name="message">`에서 **`name` 속성이 딕셔너리의 Key**가 되고 입력값이 Value가 되어 URL 끝에 `?message=안녕`(Query String)으로 날아가면, View 함수는 `request.GET.get('message')`로 안전하게 꺼냅니다. (`request.GET['message']`로 꺼내면 키가 없을 때 `MultiValueDictKeyError`로 서버가 뻗지만, `.get()` 메서드는 에러 대신 `None`을 반환해 훨씬 안전합니다!)

핵심 기능	작동 메커니즘 (How it works)	왜 이렇게 설계했을까? & 초보자 단골 실수 교정
① **`include()` 가위질** `path('articles/', include(...))`	루트 `urls.py`가 앞부분 `articles/`와 일치하는 순간 앞부분을 잘라내고, 남은 뒷부분만 `articles/urls.py`로 위임함	**[실수 주의] 후행 슬래시(Trailing Slash `/`) 필수!** 만약 `path('articles', ...)`처럼 끝에 `/`를 빼먹으면 하위 경로와 합쳐질 때 `articlesdetail/`처럼 글자가 들러붙어 404 에러가 발생함!
② **Variable Routing** `path('<int:num>/', views.detail)`	URL 경로 중 `<타입:변수명>/` 자리에 들어온 값을 낚아채어, `def detail(request, num):` 함수의 동일한 이름의 인자로 주입함	**[실수 주의]** `urls.py`에 적은 변수 이름(`<int:article_pk>`)과 `views.py` 함수의 매개변수 이름(`def detail(request, article_pk):`)이 철자 하나라도 다르면 즉시 `TypeError` 발생!

핵심 기능	작동 메커니즘 (How it works)	왜 이렇게 설계했을까? & 초보자 단골 실수 교정
③ URL 이름표 역참조 `{% url 'articles:index' %}`	앱의 `urls.py`에 성씨(`app_name = 'articles'`)와 별명(`name = 'index'`)을 붙여, 템플릿과 View(`redirect`)에서 별명으로만 호출함	주소가 `/articles/`에서 `/board/`로 바뀌어도 HTML 100개를 고칠 필요가 없음. 만약 오타로 없는 별명을 부르면 `NoReverseMatch` 에러가 뜨므로 에러 메시지만 봐도 원인을 즉시 알 수 있음!

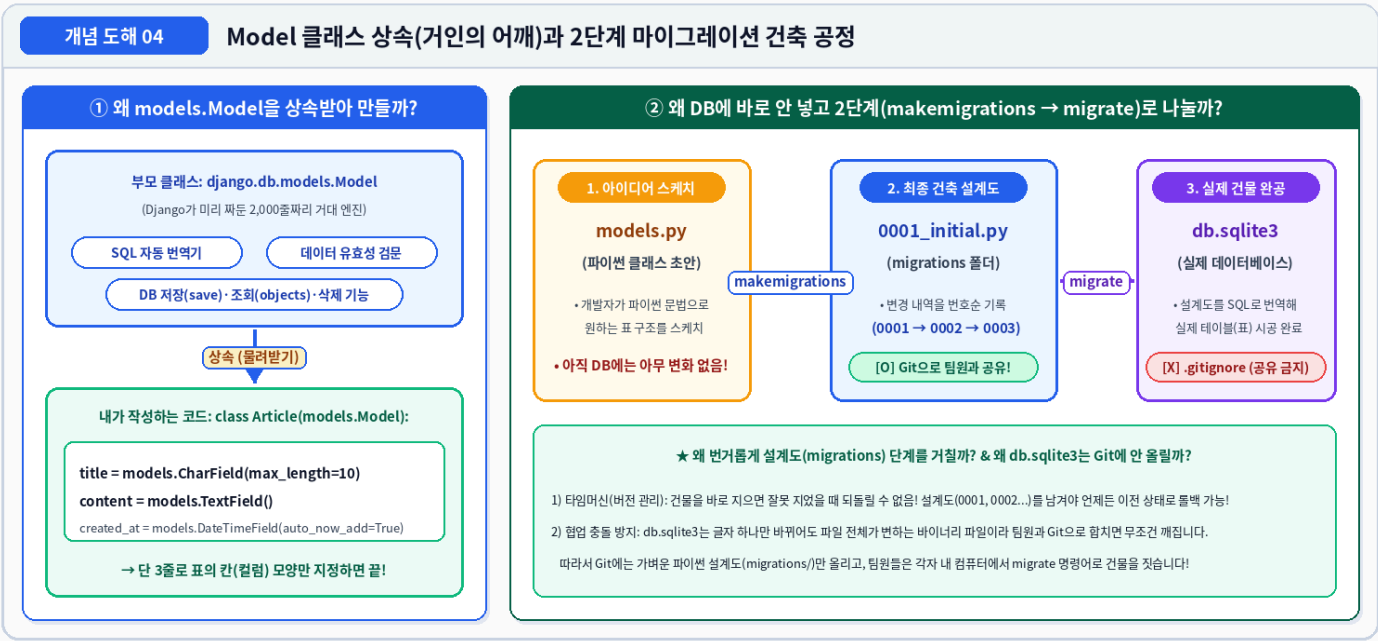
제 4 단원 : Model 클래스 상속과 2단계 마이그레이션(Migration)의 원리 (교재 3권)

1. 왜 class Article(models.Model):처럼 상속을 받아 모델을 만들까?

우리가 `models.py`에 작성하는 코드는 고작 3~4줄뿐인데 복잡한 DB 테이블을 만들고 검증할 수 있는 이유는, Django가 미리 만들어 둔 2,000줄짜리 부모 클래스 `django.db.models.Model`**을 상속(거인의 어깨에 올라타기)**받았기 때문입니다. 이때 우리가 명시적으로 적지 않아도 Django는 모든 모델에 1번부터 자동으로 1씩 증가하는 고유 번호표 컬럼인 `id`**(Primary Key,** `BigAutoField`**)**를 자동으로 생성해 줍니다.

2. 왜 2단계(makemigrations → migrate)로 나누며, 도중에 영어 질문이 뜨면 어떻게 할까?

- **1단계** `makemigrations` **(초안 → 버전별 설계도 작성) & 2단계** `migrate` **(설계도 → 실제 DB 시공)**: 건물을 지을 때 설계도 이력(`0001`, `0002`...)을 남겨두어야 나중에 기둥 설계가 잘못되었을 때 이전 버전으로 안전하게 되돌릴(Rollback) 수 있고, 무거운 `db.sqlite3` 대신 가벼운 파이썬 설계도(`migrations/`)만 Git으로 팀원과 공유할 수 있습니다.
- **[베테랑 실전 팁] 이미 게시글이 들어있는 상태에서 새 필드를 추가하고** `makemigrations`**를 쳤더니 영어로 1번/2번 선택지가 뜰 때!**
  기존에 작성된 게시글이 10개 있는데 갑자기 `models.py`에 `author = models.CharField(max_length=10)`를 추가하면, Django가 당황해서 멈추고 묻습니다: *"이미 DB에 살고 있는 기존 10개의 게시글에는 새로 생긴* ***author*** *칸에 무슨 값을 채워 넣을까요?(빈칸 불가!)"* 이때 당황하지 말고 **`1`을 입력 후 엔터**를 치고 기본값(예: `'익명'` 또는 시간 필드의 경우 `timezone.now`)을 입력해 주면 기존 행들에 기본값이 채워지며 설계도가 무사히 만들어집니다!



[그림 4] models.Model 상속의 원리와 왜 DB 반영을 2단계(makemigrations 설계도 → migrate 시공)로 나누는지 보여주는 건축 비유

헷갈리기 쉬운 개념 쌍	A의 정확한 작동 원리	B의 정확한 작동 원리 & 시니어 실전 주의사항
`auto_now_add=True` vs `auto_now=True`	`auto_now_add=True` **(최초 생성일** `created_at`**)**: 데이터가 맨 처음 생성(INSERT)될 때의 시각을 도장처럼 찍고, 이후 수정해도 절대 바뀌지 않음	`auto_now=True` **(최종 수정일** `updated_at`**)**: 객체에 `save()`가 호출될 때마다 현재 시각으로 계속 새로 덮어써짐
`null=True` vs `blank=True`	`null=True` **(DB 저장 기준)**: DB 컬럼에 `NULL` 저장 허용. 숫자·날짜·외래키(`IntegerField`, `DateTimeField` 등)를 비울 때만 사용함	`blank=True` **(Form 유효성 검사 기준)**: 입력 폼을 빈칸으로 제출해도 통과시킴. **[절대 규칙] 문자열 필드(`CharField`, `TextField`)에는** `null=True`**를 쓰지 말고 오직** `blank=True`**만 쓸 것!** (둘 다 쓰면 '값 없음'이 `NULL`과 빈 문자열 `''` 두 가지로 저장되어 데이터 조회가 꼬임!)

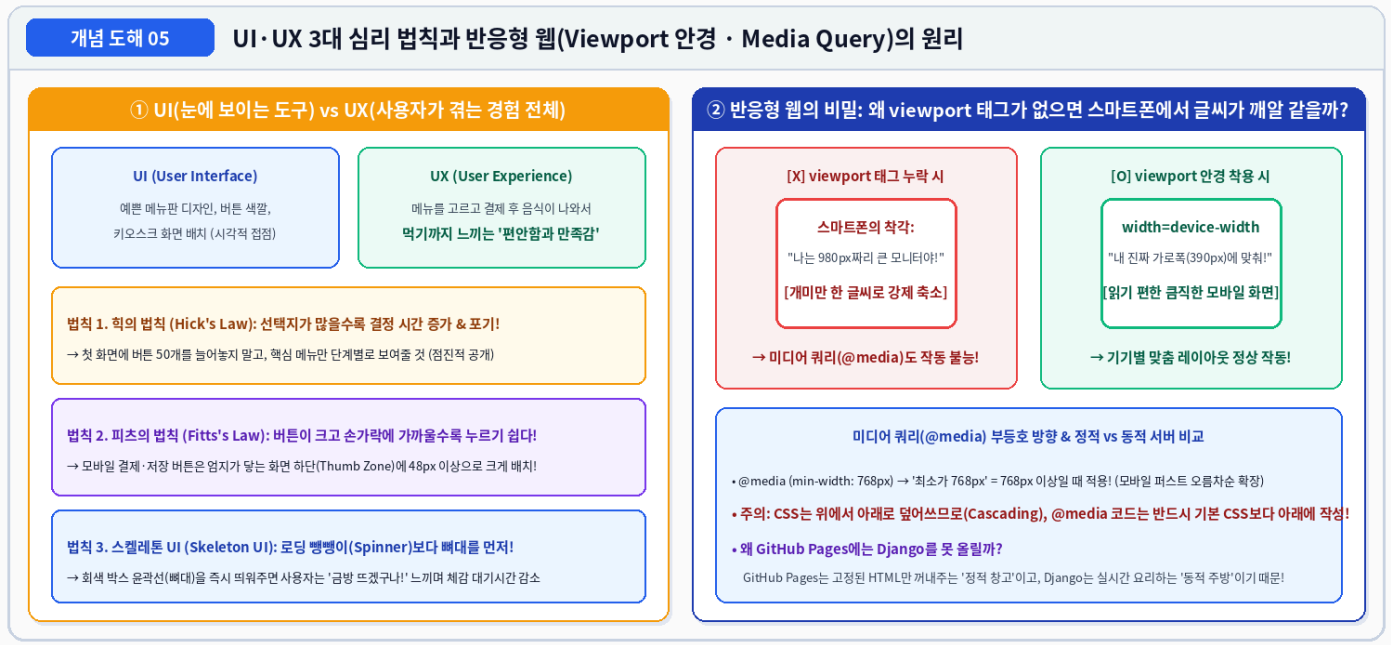
헷갈리기 쉬운 개념 쌍	A의 정확한 작동 원리	B의 정확한 작동 원리 & 시니어 실전 주의사항
`migrations/` 폴더 vs `db.sqlite3` 파일	`migrations/` 설계도 (Git 업로드 [O]): 가벼운 파이썬 텍스트 파일이므로 Git으로 팀원들과 공유하여 모두가 똑같은 테이블 구조를 갖게 함	`db.sqlite3` 데이터베이스 (`.gitignore` 제외 [X]): 데이터 한 글자만 바뀌어도 파일 전체가 변하는 바이너리 파일이라 Git 병합 시 100% 충돌이 발생하므로 절대 공유하지 않음

제 5 단원 : UI·UX 심리 법칙과 반응형 웹(Viewport · Media Query)의 원리 (교재 4권)

1. UI(도구)와 UX(경험)의 차이, 그리고 개발자가 알아야 할 3대 UX 심리 법칙

식당에 갔을 때 눈에 보이는 메뉴판의 디자인과 키오스크 버튼 색깔은 **UI(User Interface)**이고, 식당에 들어와 주문하고 식사 후 나가기까지 손님이 느끼는 총체적인 만족감과 편리함은 **UX(User Experience)**입니다.

- **힉의 법칙 (Hick's Law)**: 선택지 개수가 많을수록 결정 시간이 길어지며 이탈합니다 → 첫 화면에는 핵심 메뉴만 두고 단계별로 펼치는 **점진적 공개(Progressive Disclosure)** 적용.
- **피츠의 법칙 (Fitts's Law)**: 버튼이 크고 손가락에 가까울수록 누르기 쉽습니다 → 모바일에서는 엄지손가락이 닿는 화면 하단(**Thumb Zone**)에 최소 48×48px 이상의 큼직한 버튼 배치.
- **스켈레톤 UI & 웹 폰트 `display=swap`**: 로딩 스피너 대신 회색 뼈대(Skeleton)를 먼저 보여주어 체감 대기 시간을 줄이고, 웹 폰트 로드 시 `display=swap`을 적용해 폰트가 다 내려받아지기 전까지 기본 글꼴을 먼저 보여주어 글자가 아예 안 보이는 현상(**FOIT**)을 방지합니다.



[그림 5] UI vs UX의 3대 심리 법칙(힉의 법칙·피츠의 법칙·스켈레톤 UI)과 반응형 웹의 viewport 안경 착용 원리

2. 반응형 웹의 비밀: viewport 교정 안경과 미디어 쿼리(@media) 순서 역전 함정

- **왜 `<meta name="viewport" content="width=device-width, initial-scale=1.0">`이 필수일까?** 이 태그가 없으면 스마트폰 브라우저는 PC용 홈페이지가 깨지지 않게 하려고 **스스로를 980px짜리 데스크톱 모니터라고 거짓말**하여 화면을 개미만하게 축소해 버립니다. 이 안경을 씌워야 기기의 실제 가로 폭(예: 390px)을 정직하게 인식합니다.
- **[베테랑 디버깅 포인트] `min-width`와 `max-width`는 작성 순서(오름차순 vs 내림차순)가 정반대다!**
  CSS는 위에서 아래로 흐르며 **마지막에 참(True)이 되는 규칙이 이깁니다**. 따라서 모바일 퍼스트인 `min-width`(~이상)를 쓸 때는 반드시 **작은 숫자부터 큰 숫자 순서(`480px` → `768px` → `1024px` 오름차순)**로 아래쪽에 적어야 합니다! 만약 `min-width: 1024px`을 위에 적고 `min-width: 768px`을 아래에 적으면, 1200px짜리 PC 모니터는 둘 다 참이므로 아래에 있는 태블릿(768px) 디자인이 PC 디자인을 덮어버리는 버그가 발생합니다! (반대로 `max-width`는 큰 숫자 → 작은 숫자 내림차순으로 작성합니다.)
- **정적 호스팅(GitHub Pages) vs 동적 웹 서버(Django)**: GitHub Pages는 완성된 HTML/CSS/JS 파일만 그대로 꺼내주는 **정적(Static) 창고**이므로, 요청마다 파이썬과 DB를 실시간으로 돌리는 **동적(Dynamic) 서버**인 Django는 배포할 수 없습니다.

제 6 단원 : ORM 동시통역사와 QuerySet API의 작동 원리 (교재 5권)

1. ORM 동시통역사와 '지연 평가(Lazy Evaluation)'의 비밀

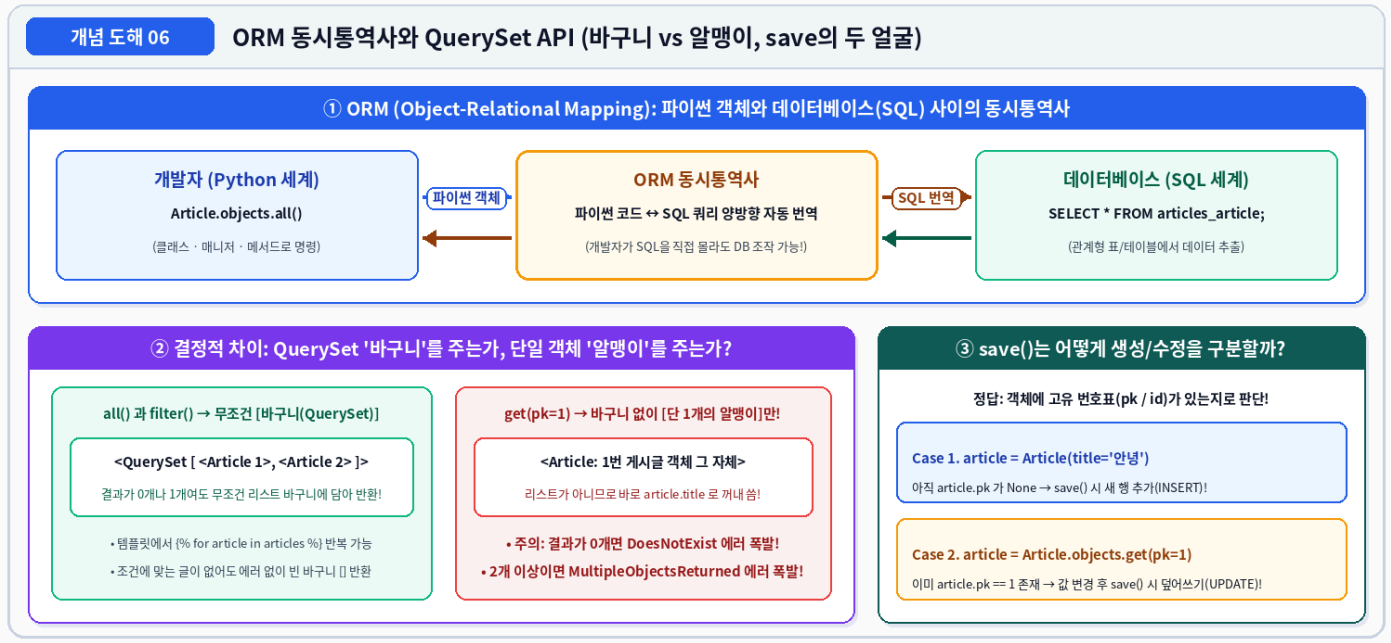
ORM(Object-Relational Mapping)은 파이썬 객체(`Article.objects.all()`)와 데이터베이스 SQL(`SELECT * FROM ...`) 사이를 양방향으로 번역해 주는 자동 동시통역사입니다. 여기서 시니어 개발자들이 가장 중요하게 여기는 ORM의 성질이 바로 **'지연 평가**

(Lazy Evaluation — 게으른 실행)'입니다.

- **QuerySet은 실제로 데이터를 쓰기 전까지는 DB에 가지 않는다!** 예를 들어 `articles = Article.objects.filter(title__contains='장고')`라고 코드를 적는 순간에는 DB에 SQL이 전혀 날아가지 않습니다. 오직 템플릿에서 `{% for article in articles %}`처럼 **실제로 상자를 열어 알맹이를 꺼내 쓰려는 순간에 딱 한 번 DB에 다녀옵니다!** 덕분에 여러 조건(`.filter().exclude().order_by('-pk')`)을 길게 이어 붙여도 DB가 느려지지 않고 마지막에 단 한 줄의 최적화된 SQL로 합쳐서 실행됩니다.

2. 시험과 실무 단골 함정: filter() 바구니 vs get() 알맹이, 그리고 save()의 원리

메서드 구분	반환되는 데이터 형태 (비유)	결과가 0개이거나 2개 이상일 때	올바른 사용처 및 베테랑 실무 팁
`all()` `filter(...)`	**QuerySet (리스트 형태의 바구니)** 결과가 1개여도 `[ <Article 1> ]`처럼 무조건 바구니에 담아 줌	• 0개일 때: 에러 없이 **빈 바구니** `<QuerySet []>` 반환 • 여러 개일 때: 모두 바구니에 담아 반환	목록 조회, 검색 등 결과가 0~N개인 모든 곳. 바구니 자체에는 `articles.title`을 쓸 수 없고 `{% for %}`로 꺼내야 함!
`get(...)`	**단일 모델 인스턴스 (알맹이 1개)** 바구니 없이 `<Article 1>` 객체 그 자체를 손에 바로 쥐여줌	• 0개일 때: **`DoesNotExist` 에러 폭발!** • 2개 이상일 때: **`MultipleObjectsReturned` 에러 폭발!**	**반드시 고유 번호(`pk` 또는 `id`) 조회에만 사용!** [실무 팁] 없는 글 번호를 조회할 때 서버가 500 에러로 죽지 않게 실무에서는 `get_object_or_404(Article, pk=pk)`를 써서 404 에러로 우아하게 안내함



[그림 6] ORM의 양방향 번역 원리와 QuerySet API에서 가장 많이 틀리는 반환 타입(QuerySet 바구니 vs 단일 인스턴스 알맹이) 비교

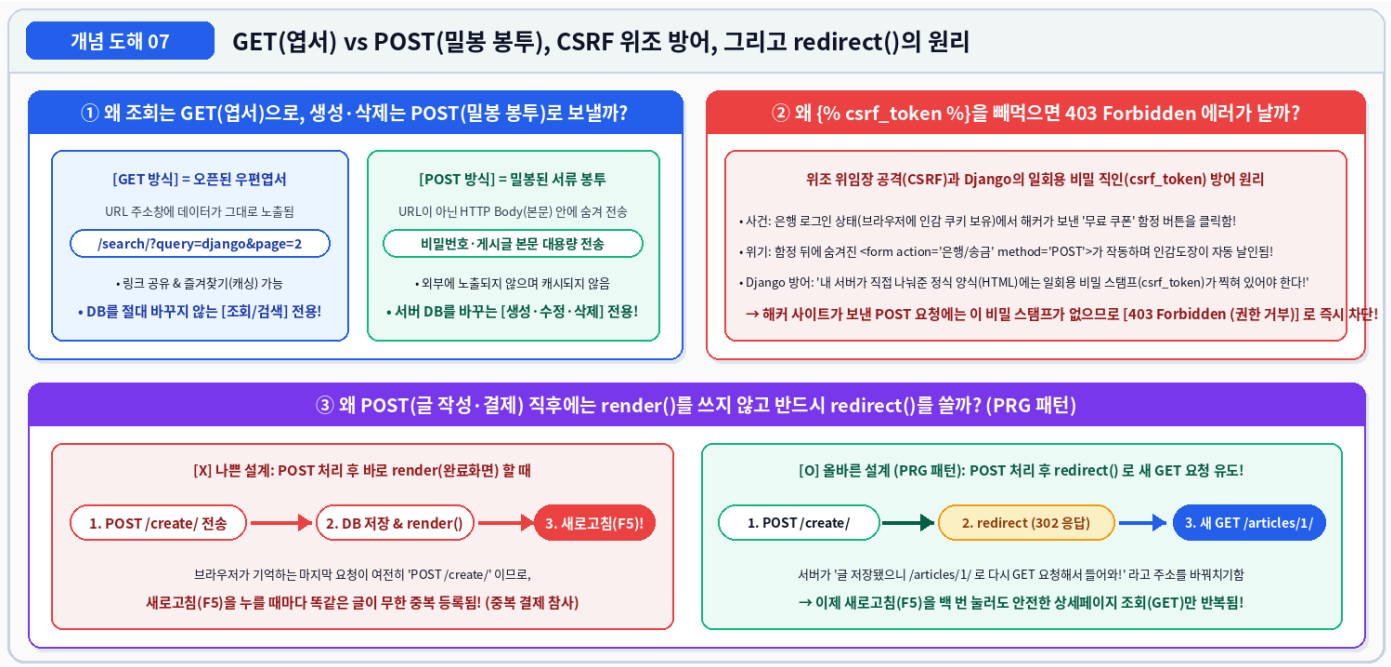
`article.save()`는 어떻게 생성(INSERT)과 수정(UPDATE)을 구분할까? 객체에 고유 번호표(`article.pk`)가 있는지로 판단합니다! 방금 메모리에 만든 새 객체는 `article.pk`가 `None`이므로 `save()` 시 새 행을 추가(`INSERT`)하고, DB에서 꺼내온 객체는 이미 `article.pk == 1` 번호표가 있으므로 값을 바꾼 뒤 `save()`를 부르면 기존 1번 행을 덮어씁니다(`UPDATE`).

제 7 단원 : HTTP GET vs POST, CSRF 방어, 그리고 redirect()의 원리 (교재 6권)

1. 왜 조회는 GET(엽서)으로, 생성·수정·삭제는 반드시 POST(밀봉 봉투)로 보낼까?

- **`GET` 방식 (오픈 우편엽서 — 멱등성·안전성 보장)**: 데이터를 URL 쿼리 스트링(`?page=2`)에 노출시켜 보내므로 링크 공유와 즐겨찾기가 가능하지만, **서버의 DB 상태를 절대 변경하지 않는 단순 조회(Read)·검색**에만 써야 합니다.
- **`POST` 방식 (밀봉 서류 봉투 — 상태 변경 전용)**: 데이터를 HTTP Body 안에 숨겨 보내며, **서버 DB를 바꾸는 생성(Create)·수정(Update)·삭제(Delete)**에 사용합니다.
- **[베테랑 보안 경고] 만약 게시글 삭제(Delete)를 편하다고 `GET` 방식(`<a href="/articles/1/delete/">`)으로 만들면 어떻게 될까?**
  `GET` 요청은 `csrf_token` 검사를 하지 않습니다! 악플러가 댓글에 `<img src="/articles/1/delete/">`라고 이미지 태그 하나만 심어두면, 관리자가 그 댓글을 눈으로 보는 순간 브라우저가 이미지를 불러오려고 자동으로 `GET /articles/1/delete/` 요청을

날려 클릭조차 안 했는데 게시글이 전부 삭제되는 대참사가 일어납니다. 그래서 삭제 버튼은 반드시 `<form method="POST">`와 `{% csrf_token %}`으로 만들어야 합니다!



[그림 7] GET(엽서) vs POST(밀봉 봉투), CSRF 위조 인감 공격과 403 Forbidden 방어, 그리고 POST 후 redirect()를 쓰는 PRG 패턴

2. CSRF 위조 방어(403)와 POST 직후 redirect()를 쓰는 PRG 패턴, 그리고 상태 코드 진단

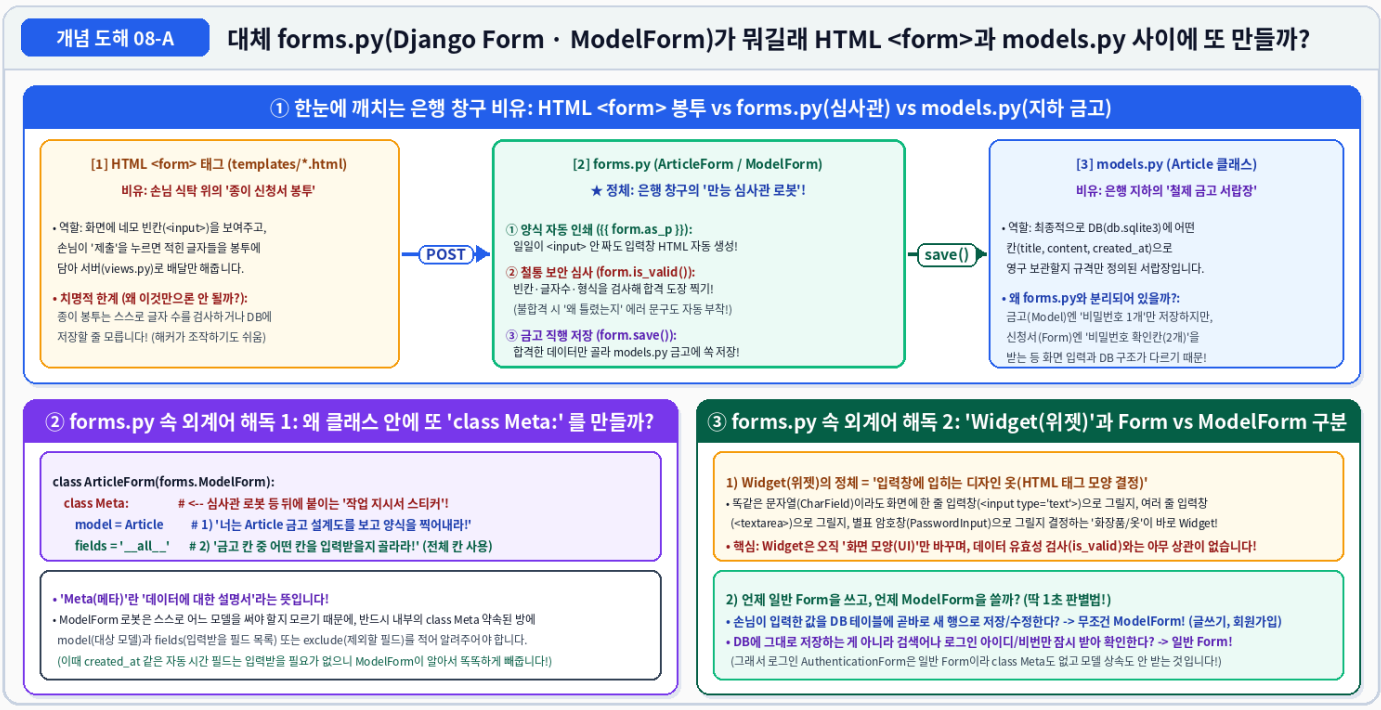
해커가 사용자의 브라우저에 저장된 로그인 쿠키(인감도장)를 악용해 몰래 송금·글쓰기 요청을 보내는 **CSRF(사이트 간 요청 위조)**를 막기 위해, Django는 정식 양식에만 일회용 비밀 스탬프(`{% csrf_token %}`)를 찍어주고 스탬프가 없는 `POST` 요청은 `403 Forbidden`으로 차단합니다. 또한 `POST`로 글을 저장한 직후 바로 `render()`를 하면 새로고침(F5) 시 마지막 `POST` 요청이 재전송되어 글이 무한 중복 등록되므로, 반드시 `redirect()`(`302 Found`)로 브라우저가 새로운 `GET` 요청을 보내도록 유도해야 합니다(**Post-Redirect-Get 패턴**).

HTTP 상태 코드	의미 및 발생 상황	베테랑 개발자의 10초 원인 진단 & 해결책
`200 OK` / `302 Found`	200: 정상 렌더링(`render`) 302: 다른 주소로 이동 명령(`redirect`)	터미널 로그에 `POST /create/ 302` 직후 `GET /articles/1/ 200`이 찍히면 PRG 패턴이 완벽하게 작동한 정상 상태!
`403 Forbidden`	서버가 요청을 이해했으나 권한 거부	**99% 확률로 `<form method="POST">` 바로 아래에 `{% csrf_token %}`을 빼먹었을 때 발생!**
`404 Not Found`	요청한 URL 경로 또는 데이터가 존재하지 않음	`urls.py`의 경로 오타, 후행 슬래시(`/`) 누락, 혹은 삭제된 게시글 번호 조회
`500 Server Error`	백엔드 서버 내부 코드 폭발 (개발자 과실!)	`views.py`나 `models.py`의 파이썬 문법 오타, `get()`으로 없는 번호 조회, `render` 템플릿 경로 오타

제 8 단원 : forms.py(Form · ModelForm)의 진짜 정체와 단일 View 통합의 원리 (교재 7권)

0. [기초부터 깨치기] 대체 forms.py(Django Form)가 뭐길래 HTML <form>과 models.py 사이에 또 만들까?

입문자가 가장 많이 하는 질문이 바로 **"이미 화면에는 HTML `<form>` 태그가 있고 DB에는 `models.py`가 있는데, 왜 굳이 중간에 `forms.py`라는 파일을 또 만들어서 어렵게 쓰나요?"**입니다. 결론부터 말하면, HTML `<form>`은 데이터만 담아 나르는 **'빈 종이봉투'**에 불과해서 스스로 불량 데이터를 걸러내지 못하기 때문에, 중간에서 **① 입력창 HTML을 자동으로 찍어내고 + ② 데이터 유효성을 철통 심사(`is_valid()`)한 뒤 + ③ DB 금고(`models.py`)에 안전하게 넣어주는 '은행 창구 심사관 로봇(`forms.py`)'**을 두는 것입니다!



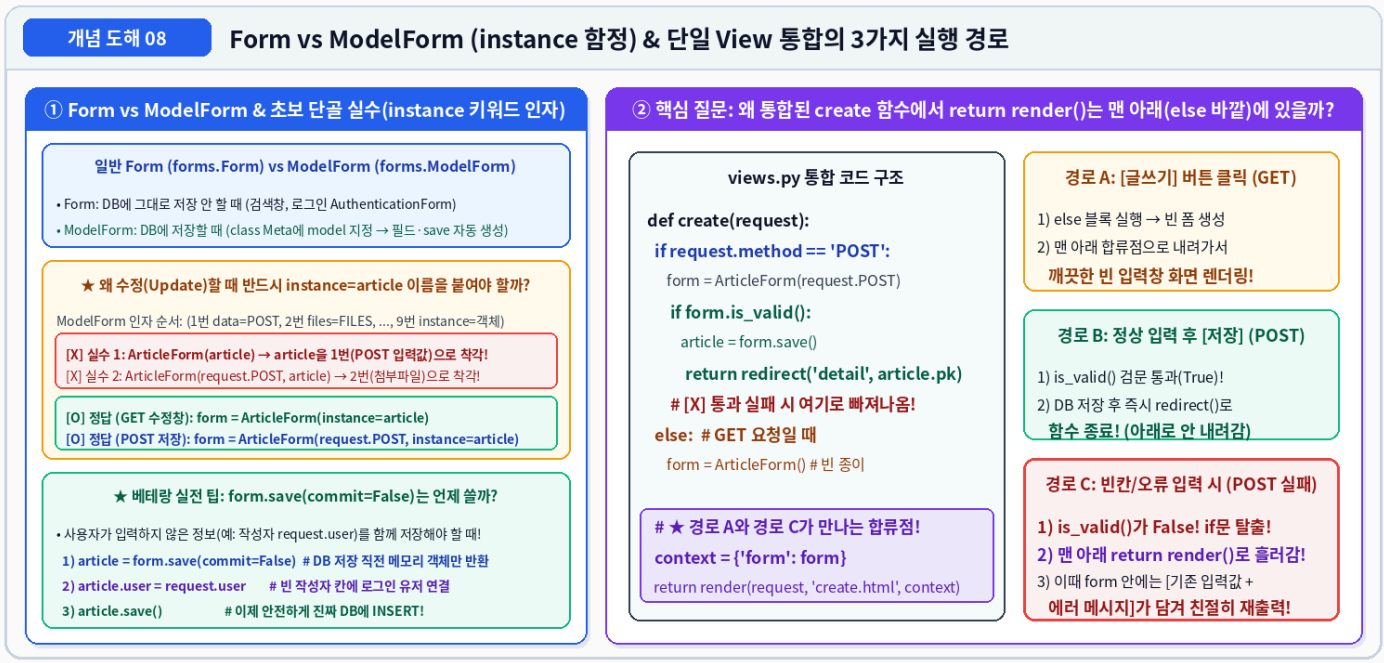
[개념 도해 08-A] 은행 창구 비유로 깨치는 HTML <form>(종이봉투) vs forms.py(창구 심사관 로봇) vs models.py(지하 금고), 그리고 class Meta와 Widget 해독

헷갈리는 3대 요소	은행 창구 비유 (정체)	실제 하는 일 & 왜 서로 분리되어 있어야 할까?
① HTML `<form>` 태그 (`templates/*.html`)	손님 식탁 위에 놓인 **'종이 신청서 봉투'**	브라우저 화면에 네모난 입력칸을 보여주고, 손님이 '제출(Submit)' 버튼을 누르면 적힌 글자들을 봉투에 담아 서버(`views.py`)로 배달만 해줍니다. **단, 종이봉투는 스스로 글자 수를 검사하거나 DB에 저장할 능력이 전혀 없습니다!**
② Django `forms.py` (`Form` / `ModelForm`)	은행 창구에 앉아 있는 **'만능 심사관 로봇'**	**[3대 핵심 마법]** ① `{{ form.as_p }}`로 HTML `<input>` 양식을 자동 인쇄해 주고, ② 손님이 낸 봉투에 빈칸이나 글자 수 초과가 없는지 **`form.is_valid()`로 엄격히 심사**(틀리면 에러 메시지도 자동 부착!)하며, ③ 합격한 데이터만 **`form.save()`로 DB 금고에 넣어줍니다!**
③ Django `models.py` (`Article`, `User` 클래스)	은행 지하에 있는 **'철제 금고 서랍장'**	최종적으로 데이터베이스(`db.sqlite3`)에 어떤 칸(`title`, `content`, `created_at`)으로 영구 보관할지 규격만 정의된 서랍장입니다. 예를 들어 금고(Model)에는 '비밀번호 1개'만 저장하지만 신청서(Form)에서는 '비밀번호 확인칸 2개'를 입력받아야 하므로 두 파일의 역할을 분리합니다.
`forms.py` 속 외계어 해독: `class Meta` & `Widget`	**`Meta` = '로봇 작업 지시서'** **`Widget` = '입력창 디자인 옷'**	• **`class Meta:`**는 **`ModelForm`** 로봇 등 뒤에 붙이는 스티커로, *"너는 **`model = Article`** 금고 설계도를 보고 **`fields = '__all__'`**(모든 칸)의 입력 양식을 찍어내라!"*고 알려주는 약속된 설정 방입니다. • **`Widget(위젯)`**은 데이터 검사 규칙과는 무관하게, 화면에 한 줄 입력창(`TextInput`)으로 그릴지 여러 줄(`Textarea`)로 그릴지 **HTML 모양(옷)만 꾸며주는 도구**입니다.

1. 언제 Form을 쓰고 언제 ModelForm을 쓰며, 수정할 때 왜 instance=article을 꼭 붙일까?

프론트엔드의 **`<input required>`**는 사용자가 크롬 개발자 도구(F12)로 지우면 뚫리기 때문에, 반드시 백엔드 검문소인 **Django Form(`form.is_valid()`)**으로 한 번 더 검증해야 합니다. 이때 DB에 곧바로 저장하지 않는 로그인·검색창은 **`forms.Form`**을 쓰고, DB 테이블에 곧바로 저장하는 게시글·회원가입은 **`forms.ModelForm`**을 사용합니다.

- **[초보자 실수 1위] 왜 수정 화면에서 `ArticleForm(article)`이라고 쓰면 에러가 날까?**
  `ModelForm`의 괄호 안 매개변수 순서는 **1번이 `data`(사용자 입력값 `request.POST`), 2번이 `files`(첨부파일 `request.FILES`), 그리고 저 뒤쪽 9번째가 `instance`(기존 DB 객체)**입니다! 따라서 키워드 이름 없이 **`ArticleForm(article)`**이라고 적으면 Django는 **`article`** 객체를 사용자가 방금 제출한 **`POST`** 데이터(1번 인자)로 착각해 버리고, **`ArticleForm(request.POST, article)`**이라고 적으면 첨부파일(2번 인자)로 착각해 새 글(INSERT)을 하나 더 만들어 버립니다! 기존 글을 수정할 때는 반드시 **`instance=article`**이라고 이름표를 명시해야 합니다.



[그림 8] Form vs ModelForm(instance 위치 인자 함정 · commit=False)과 통합 create 함수의 3가지 실행 경로 분석

2. 왜 new와 create를 하나로 합칠 때 return render()가 맨 아래(else 바깥)에 있을까?

```
def create(request):
    if request.method == 'POST':                # [분기 1] 사용자가 내용을 채워 '저장' 버튼을 눌렀을 때
        form = ArticleForm(request.POST)        # 사용자가 입력한 데이터를 채운 폼 생성
        if form.is_valid():                     # 검문소 통과 시 (경로 B)
            article = form.save()
            return redirect('articles:detail', article.pk)  # 즉시 함수 종료! 아래로 내려가지 않음!
        # ★ 만약 is_valid()가 실패(False)하면? return을 못 만나고 if문 바깥(맨 아래)으로 미끄러져 내려감! (경로 C)
    else:                                       # [분기 2] 사용자가 '글쓰기' 링크를 눌러 처음 들어왔을 때 (GET)
        form = ArticleForm()                    # 깨끗한 빈 종이 폼 생성 (경로 A)
    # ★ 경로 A(빈 폼)와 경로 C(기존 입력값 + 에러 메시지가 담긴 폼)가 만나는 공동 출구!
    context = {'form': form}
    return render(request, 'articles/create.html', context)
```

만약 마지막 두 줄을 탭(Tab)을 눌러 `else:` 블록 안으로 넣어버리면, 사용자가 제목을 비워두어 `form.is_valid()`가 실패했을 때(경로 C) 함수가 아무것도 반환하지 못하고(`ValueError: didn't return an HttpResponse object`) 서버가 뻗어버립니다! 맨 아래 `render()`가 바깥에 버티고 있어야만, 유효성 검사에 실패한 `form` 객체가 **'사용자가 아까 열심히 친 본문 + 친절한 에러 메시지'**를 품은 채 화면에 다시 그려집니다.

제 9 단원 : 쿠키·세션(sessionid)과 Django 인증 시스템의 원리 (교재 8·9권)

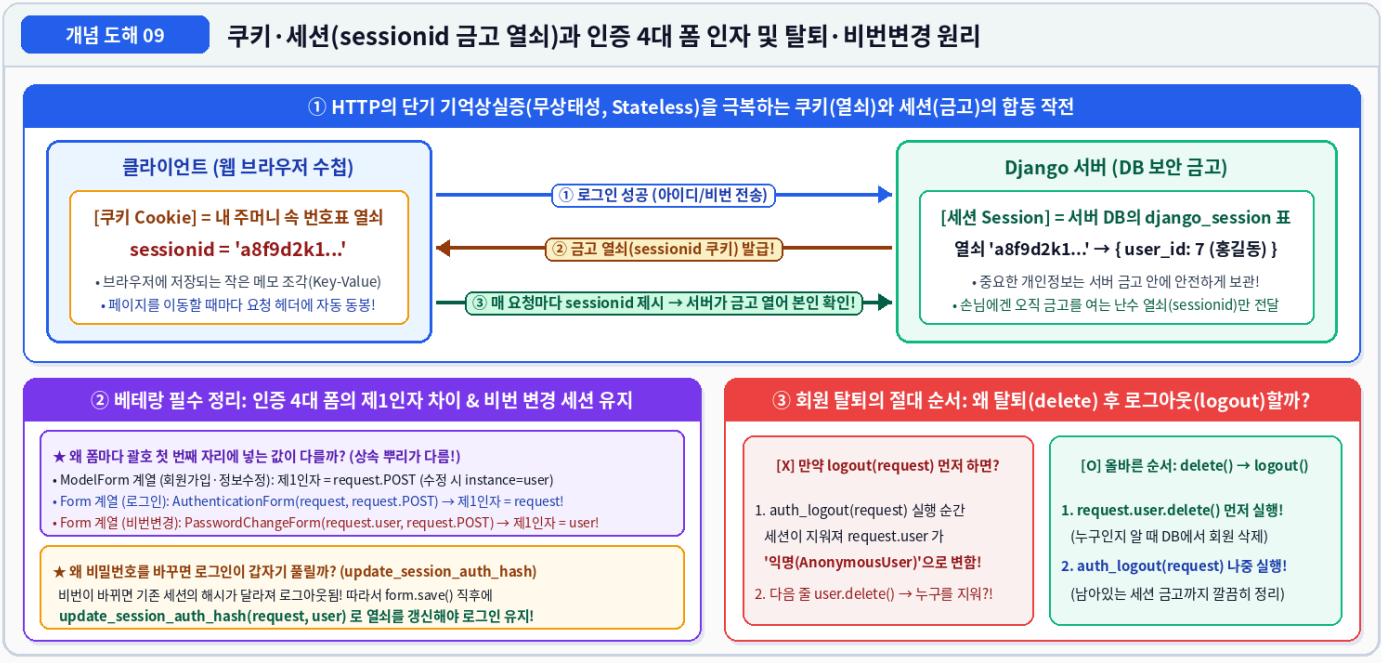
1. HTTP의 단기 기억상실증(무상태성)을 극복하는 쿠키(열쇠)와 세션(금고)

HTTP는 응답 직후 연결을 끊고(비연결성) 이전 요청을 기억하지 않는(**무상태성, Stateless**) 프로토콜입니다. 그래서 로그인(`auth_login(request, user)`)에 성공하면 Django 서버는 자신의 DB(`django_session` 테이블)라는 **보안 금고**에 로그인 정보를 넣고, 브라우저의 **쿠키(Cookie)**에는 오직 그 금고를 여는 난수 열쇠 번호(`sessionid`)만 건네줍니다. 이후 브라우저가 매 요청마다 `sessionid` 열쇠를 제시하면 서버가 금고를 열어 `request.user`를 알아봅니다.

2. 베테랑 개발자의 핵심 정리: 초보자가 100% 헷갈리는 '인증 4대 폼 인자 순서'와 필수 원리

회원가입, 로그인, 회원정보 수정, 비밀번호 변경을 구현할 때 초보자들이 가장 많이 겪는 에러는 **"어떤 폼은 첫 번째 인자로 `request.POST`를 넣고, 어떤 폼은 `request`나 `request.user`를 넣는다"**는 점입니다. 그 이유는 각 폼이 물려받은 부모 클래스(`ModelForm` vs `Form` vs `SetPasswordForm`)가 다르기 때문입니다!

기능 및 사용 폼 이름	상속 뿌리	POST 요청 시 올바른 인자 전달 모양	왜 인자 순서가 이렇게 생겼을까? (원리)
① 회원가입 `CustomUserCreationForm`	`ModelForm`	`form = CustomUserCreationForm(request.POST)`	새 회원을 User 테이블에 `INSERT`하는 `ModelForm`이므로 제1인자가 `data`(`request.POST`)임 (커스텀 User 교체를 위해 상속 필수!)
② 회원정보 수정 `CustomUserChangeForm`	`ModelForm`	`form = CustomUserChangeForm(request.POST, instance=request.user)`	기존 회원 행을 `UPDATE`하는 `ModelForm`이므로 제1인자는 `request.POST`, 수정 대상은 키워드 인자 `instance=request.user`로 전달!
③ 로그인 `AuthenticationForm`	일반 `Form`	`form = AuthenticationForm(request, request.POST)`	DB에 새 행을 만드는 것이 아니라 세션 인증을 처리하는 일반 `Form`이므로, 제1인자로 반드시 `request`를 받고 제2인자로 `request.POST`를 받음!
④ 비밀번호 변경 `PasswordChangeForm`	`SetPasswordForm` (일반 `Form`)	`form = PasswordChangeForm(request.user, request.POST)`	부모 클래스인 `SetPasswordForm`의 생성자 규칙에 따라 누구의 비밀번호를 바꿀지인 대상 회원 객체(`request.user`)를 제1인자로 받음!



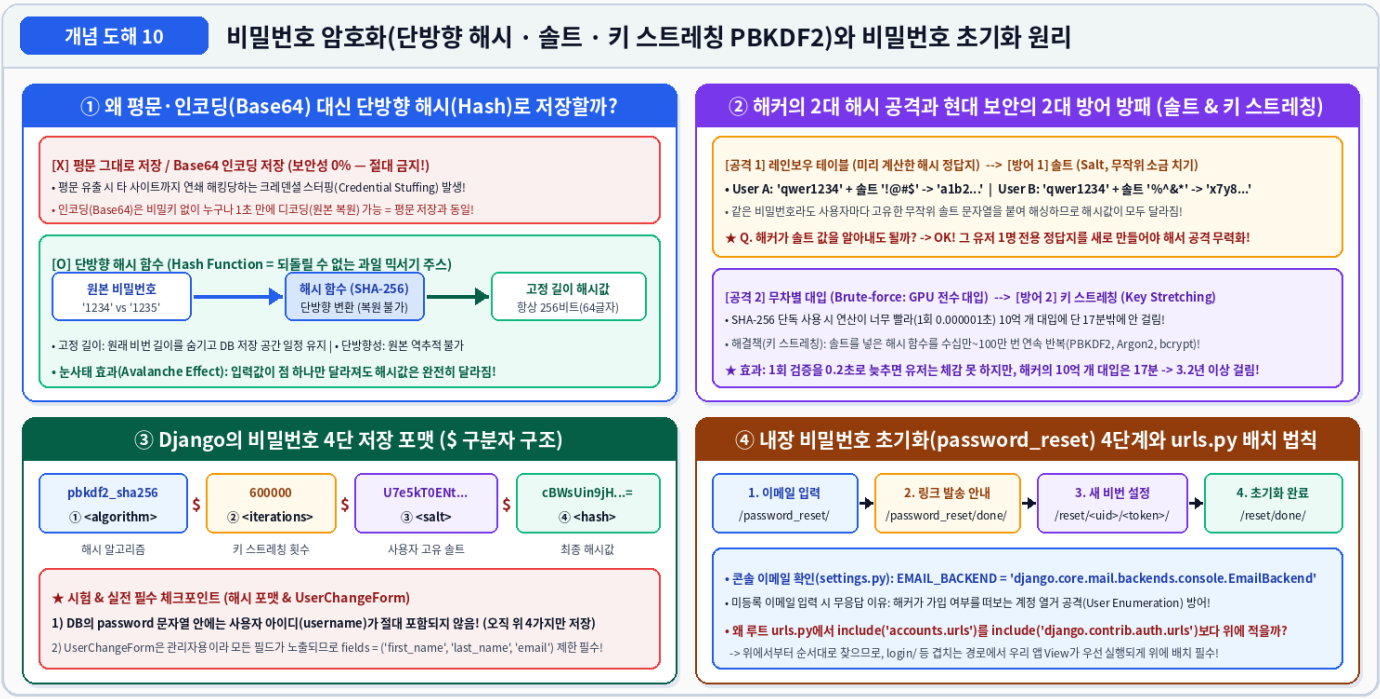
[그림 9] 쿠키(번호표 열쇠)와 세션(서버 금고) 합동 작전, 인증 4대 폼 인자 규칙, 비번 변경 세션 유지 및 회원 탈퇴 순서

- **왜 첫 번째 `migrate` 전에 `AbstractUser`로 User 모델을 갈아끼울까?** 기본 `auth.User`는 나중에 닉네임 같은 컬럼을 추가하기 어렵고, 운영 도중에 모델을 바꾸면 이미 연결된 모든 외래키(FK)와 마이그레이션이 깨지므로 첫 시공 전에 `class User(AbstractUser): pass`와 `AUTH_USER_MODEL = 'accounts.User'`로 교체해 두는 것이 철칙입니다.
- **왜 회원 탈퇴 시 `delete()`를 먼저 하고 `logout()`을 나중에 할까?** `auth_logout(request)`를 먼저 하면 세션이 지워져 `request.user`가 '익명 손님(`AnonymousUser`)'이 되어버리므로 누구를 DB에서 지울지 잃어버립니다! 반드시 ① `request.user.delete()`(계정 삭제) → ② `auth_logout(request)`(세션 청소) 순서를 지켜야 합니다.

제 10 단원 : 비밀번호 암호화(해시·솔트·키 스트레칭)와 회원수정·비번초기화 원리 (교재 10권)

1. 왜 비밀번호를 평문이나 인코딩(Base64)으로 저장하면 안 되고 '단방향 해시'로 저장할까?

- **평문 저장과 인코딩(Base64)의 치명적 위험**: DB가 해킹당했을 때 비밀번호가 평문으로 저장되어 있으면, 대부분의 사용자는 여러 사이트에서 동일한 아이디·비밀번호를 재사용하므로 다른 금융·포털 사이트까지 연쇄 해킹당하는 **크레덴셜 스터핑(Credential Stuffing) 공격**이 발생합니다. 또한 Base64 같은 **인코딩(Encoding)**은 암호화가 아니라 단순한 표현 형식 변환이어서 비밀키 없이 누구나 1초 만에 디코딩(원본 복원)할 수 있으므로 평문 저장과 똑같이 위험합니다.
- **단방향 해시(Hash = 과일 믹서기 주스 비유)**: 양방향 열쇠로 되돌릴 수 있는 암호화(Encryption)와 달리, **해시(Hash)**는 과일을 갈아 주스로 만드는 것처럼 **원본으로 절대 되돌릴 수 없는 단방향 변환**입니다. 해시 함수는 ① 어떤 길이의 입력을 넣어도 **고정된 길이**(예: `SHA-256`은 64글자)로 변환하여 원래 비밀번호 길이를 숨기고 DB 공간을 일정하게 유지하며, ② 결과만 보고 원본을 알 수 없는 **단방향성**을 지니고, ③ 입력값이 점 하나만 달라져도 결과가 완전히 바뀌는 **눈사태 효과(Avalanche Effect)**를 가집니다.



[그림 10] 단방향 해시(SHA-256)와 해커의 2대 공격(레인보우 테이블 · 무차별 대입)을 막는 솔트(Salt) · 키 스트레칭(PBKDF2) 및 비밀번호 초기화 4단계

2. 해커의 2대 해시 공격과 현대 보안의 2대 방어 방패: 솔트(Salt) & 키 스트레칭(PBKDF2)

- **[공격 1] 레인보우 테이블(Rainbow Table) ↔ [방어 1] 솔트(Salt, 무작위 소금 치기)**: 해시 함수는 같은 값을 넣으면 항상 같은 해시값이 나오므로, 해커는 자주 쓰이는 비밀번호 수억 개(`123456`, `qwer1234` 등)의 해시값을 미리 계산해 둔 거대한 정답지 사전(**레인보우 테이블**)과 대조해 비밀번호를 알아냅니다. 이를 막기 위해 각 사용자마다 무작위로 생성된 고유 문자열인 **솔트(Salt)**를 비밀번호에 덧붙여 해싱합니다. 눈사태 효과 때문에 두 사용자가 똑같이 `qwer1234`를 써도 솔트가 달라 해시값이 완전히 달라지며, 설령 해커가 DB에서 솔트 값을 함께 훔쳐보더라도 그 유저 한 명만을 위한 새 레인보우 테이블을 처음부터 다시 만들어야 하므로 공격 효율이 무력화됩니다!
- **[공격 2] 무차별 대입 공격(Brute-force) ↔ [방어 2] 키 스트레칭(Key Stretching)**: 솔트 때문에 정답지를 못 쓰게 된 해커는 최신 GPU의 초고속 연산 속도(초당 수백억 번)를 무기로 모든 비밀번호 조합을 하나씩 직접 대입(**Brute-force**)합니다. `SHA-256`은 연산이 너무 빨라(1회 0.000001초) 10억 개 대입에 단 17분밖에 안 걸립니다! 이를 막기 위해 해시 연산을 수십만~100만 번 연속 반복해 1회 검증 시간을 약 **0.2초**로 늦추는 **키 스트레칭(`PBKDF2`, `Argon2`, `bcrypt`)**을 적용합니다. 정상 사용자는 로그인할 때 0.2초를 전혀 체감하지 못하지만, 해커가 10억 개를 대입하는 시간은 **17분에서 약 3.2년 이상**으로 폭증하여 공격이 불가능해집니다.
- **Django의 비밀번호 4단 저장 포맷과 시험 단골 함정**: Django는 기본적으로 `PBKDF2` 방식을 사용하여 DB의 `password` 컬럼에 `<algorithm>$<iterations>$<salt>$<hash>` 형식(예: `pbkdf2_sha256$600000$U7e5k...$cBWsU...=`)으로 저장합니다. **[시험 주의] 이 문자열 안에 사용자 아이디(`username`)는 절대 포함되지 않습니다!**

3. 회원정보 수정(UserChangeForm) · 세션 무효화 방지 · 비밀번호 초기화의 숨은 원리

- **왜 `UserChangeForm`을 그대로 쓰면 보안 사고가 날까?**: `UserChangeForm`은 원래 관리자 페이지용으로 만들어져 `fields = '__all__'`로 설정되어 있습니다. 이를 일반 회원에게 그대로 출력하면 관리자 권한 체크박스(`is_superuser`, `is_staff`)까지 노출되어 권한 탈취 사고가 일어날 수 있으므로, 반드시 `CustomUserChangeForm`의 `Meta`에서 `fields = ('first_name', 'last_name', 'email')`처럼 안전한 필드만 지정해야 합니다. 또한 이 폼 하단에는 비밀번호 변경 링크가 기본 경로인 `/accounts/password/`로 안내되어 있습니다.

- **왜 비밀번호를 바꾸면 모든 세션이 무효화될까? (`update_session_auth_hash`)**: Django 세션에는 비밀번호 해시에서 만든 조각이 함께 기록됩니다. 비밀번호를 바꾸면 이 값이 달라져 기존 세션이 모두 로그아웃되는데, 이는 **내 계정을 몰래 쓰던 해커의 다른 기기 세션을 한꺼번에 강제 로그아웃시키는 훌륭한 보안 효과**를 가집니다! 단, 비밀번호를 바꾼 내 브라우저까지 로그아웃되면 불편하므로 `update_session_auth_hash(request, user)`로 내 현재 세션만 새 해시로 갱신해 주는 것입니다.
- **내장 비밀번호 초기화(`django.contrib.auth.urls`)와 2가지 베테랑 포인트**: 비밀번호를 잊어버렸을 때 이메일로 재설정 링크를 보내주는 기능은 루트 `urls.py`에 `path('accounts/', include('django.contrib.auth.urls'))`를 추가하고 `settings.py`에 `EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'`를 설정해 구현합니다. 이때 ① `login/` 등 겹치는 경로에서 우리 앱의 View가 먼저 실행되도록 반드시 `include('accounts.urls')`**를 내장** `auth.urls`**보다 윗줄에 적어야 하며**, ② 가입되지 않은 이메일을 입력해도 해커가 이메일 가입 여부를 떠보는 **계정 열거 공격(User Enumeration)**을 막기 위해 에러 메시지 없이 동일한 완료 화면을 보여준다는 보안 원리까지 기억하면 완벽합니다!

제 11 단원 : Model · Form · Auth 실전 심화와 초보자가 가장 헷갈리는 6대 원리 (교재 11권)

1. 자판기 비유로 깨치는 choices와 get_FOO_display(), 그리고 왜 PositiveIntegerField로 이중 방어를 할까?

- **자판기 상품 코드 비유 (`choices` & `get_FOO_display()`)**: 자판기 밖의 손님은 `'콜라'`, `'할 일'`, `'높음'`이라는 친절한 한글 버튼을 보고 누르지만, 자판기 내부 기계(DB)는 `'TODO'`, `1`, `2`, `3` 같은 간결하고 규칙적인 코드로 저장해야 데이터 일관성이 유지됩니다. 이때 `choices` 옵션을 준 필드에는 Django가 자동으로 `get_필드이름_display()`라는 번역기 메서드를 만들어 줍니다! 따라서 템플릿에서 `{{ todo.priority }}`라고 쓰면 숫자 `3`이 나오지만, `{{ todo.get_priority_display }}`(DTL이므로 괄호 생략!)라고 쓰면 `'높음'`이라는 한글 이름이 예쁘게 출력됩니다.
- **왜 `choices`로 이미 1·2·3만 고르게 해놓고 굳이 `PositiveIntegerField`(0 이상 정수)를 또 쓸까?**: `choices`**는 오직 웹 화면(Form·Model 검증 단계)에서만 확인하는 '정문 안내원'**이라, 누군가 파이썬 코드나 SQL로 DB에 직접 음수(`-5`)를 넣으려 하면 막지 못합니다! 반면 `PositiveIntegerField`**는 실제 DB 테이블 자체에** `CHECK ("priority" >= 0)`**이라는 '금고 자물쇠'**까지 걸어주므로, 검증 단계(1차)와 DB 단계(2차)의 완벽한 **이중 방어**가 완성됩니다.



[그림 11] 자판기 choices(get_FOO_display) · 공항 검색대 cleaned_data · 인테리어(Meta.widgets) vs 가구 교체(필드 재선언) · 포스트잇 messages · 안전한 ?next= 복귀

2. 공항 보안 검색대(cleaned_data)와 다중 체크박스(MultipleChoiceField) 콤마 포장의 비밀

- **왜 `request.POST`를 바로 안 쓰고 `form.cleaned_data`를 꺼내 쓸까? (공항 검색대 비유)**: 사용자가 제출한 `request.POST`나 `request.GET`은 공항에 막 도착한 승객의 가방과 같아서, 숫자를 입력했어도 **전부 단순 문자열(`'3'`)**로 들어있고 위험한 값이 섞여 있을 수 있습니다. 이 가방이 보안 검색대인 `if form.is_valid():`를 통과하는 순간, Django는 값들을 깨끗하게 소독하고 **알맞은 파이썬 자료형(정수 `3`, 날짜 객체, 리스트 `['WORK', 'STUDY']`)으로 자동 변환하여 `form.cleaned_data` 바구니에 담아줍니다!**
- **왜 다중 체크박스(`MultipleChoiceField`)를 저장할 때 `','.join()`으로 묶고 `__init__`에서 다시 풀까?**: 체크박스로 여러 개를 고르면 `cleaned_data['tags']`는 파이썬 리스트 `['WORK', 'STUDY']`가 됩니다. 그런데 DB의 `CharField`는 문자열 칸이라서 그냥 저장하면 대괄호까지 통째로 글자(`"['WORK', 'STUDY']"`)가 되어 들어갑니다! 그래서 ① **저장할 때(`views.py`)**는 `todo = form.save(commit=False)`로 멈춘 뒤 `','.join(...)`으로 `'WORK,STUDY'` 문자열로 엮어서 저장하고, ② **수정 화면을 열 때(`forms.py`의 `__init__`)**는 DB의 `'WORK,STUDY'`를 `split(',')`으로 다시 리스트로 쪼개어 `self.initial['tags']`에 넣어

주어야 기존 체크 표시가 복원됩니다. (단, 콤마 문자열 저장은 '송장 한 장에 여러 택배를 적어둔 임시방편'이라 검색·수정이 어려우며, 다음 단원인 **모델 관계 M:N**에서 근본적으로 해결합니다!)

3. 인테리어 시공(Meta.widgets) vs 가구 새로 사기(필드 재선언), 그리고 Auth 심화 4대 함정

실전 심화 핵심 주제	초보자가 가장 많이 겪는 에러 / 문제 상황	베테랑의 원리 해독 & 표준 해결 공식
① `Widget` 지정 2가지 방법 (인테리어 vs 가구 교체)	회원가입 폼(`CustomUserCreationForm`)을 꾸밀 때 `Meta.widgets`에 `password1`을 넣었더니 전혀 적용되지 않음!	• **`username`은 User 모델에 있는 필드**이므로 `Meta.widgets`(방 벽지 인테리어)로 꾸밈 (기존 모델 설정 유지!) • **`password1`·`password2`는 모델 필드가 아닌 폼 전용 칸**이므로 클래스 상단에 필드를 **재선언(가구 새로 사기)**해야 하며, 이때 사라지는 `label='비밀번호'`도 꼭 다시 적어주어야 함!
② **일회용 알림 포스트잇** (`Django Message Framework`)	글 작성 후 `redirect()`를 하면 `context`를 못 넘기는데, 어떻게 다음 화면에 *"추가 완료!"* 알림창을 띄울까?	`redirect` 직전에 `messages.success(request, '문구')`로 손님에게 '읽으면 떼는 1회용 포스트잇'을 붙여 보내고, `base.html`에서 `{% for message in messages %}`로 출력함 (`error` 등급과 부트스트랩 `danger` 색상은 `MESSAGE_TAGS`로 매칭!)
③ `?next=` **복귀와 보안 검문소** (오픈 리다이렉트 방어)	로그인 후 `request.GET.get('next')`로 바로 이동시켰더니, 해커가 `?next=https://evil.com` 링크로 사용자를 납치함!	• 택시 기사 검문소 함수인 `url_has_allowed_host_and_scheme(next_url, allowed_hosts={request.get_host()})`로 우리 사이트 주소일 때만 이동! • 템플릿 `login.html`의 `action`에는 `?next={{ request.GET.next|urlencode }}`를 써서 원래 주소 안의 `&` 기호가 잘리지 않게 보호!
④ **커스텀 User 3대 사고 예방** (`E304` · `UserAdmin` · `user_profile`)	• `fields.E304 (user_set 충돌)` 에러 • 관리자 페이지에서 비번 변경 후 로그인 불가 • 프로필 페이지 접속 시 내 로그인 정보가 바뀜	• `User` 클래스 정의 후 `settings.py`에 `AUTH_USER_MODEL`을 안 적으면 기본 User와 `user_set` 이름이 겹쳐 폭발함! • `admin.py`에 `UserAdmin` 없이 `register(User)`만 하면 비번이 해시 없이 평문 저장되어 로그인 불가! • 프로필 뷰에선 기본 `{{ user }}`(로그인한 나)와 겹치지 않게 `{'user_profile': user}`로 전달!

총괄 부록 : 베테랑 개발자의 1~11권 핵심 원리 & 단골 에러 10초 자가진단 총정리 표

초보 개발자와 베테랑 개발자의 차이는 에러가 안 나는 것이 아니라, 에러 화면이나 이상 동작이 발생했을 때 **어느 파일의 어떤 원리를 점검해야 하는지 즉시 떠올리는 것**입니다. 전 11권의 핵심 원리를 아래 표로 최종 점검해 보세요.

실전 상황 / 에러 메시지	왜 이런 현상이 발생했을까? (근본 원리)	베테랑의 10초 해결책 & 관련 단원
`TemplateDoesNotExist: base.html`	`APP_DIRS: True`는 등록된 앱 내부의 `templates/`만 탐색하며 최상위 폴더는 모르기 때문	`settings.py`의 `TEMPLATES` 안 `'DIRS': [BASE_DIR / 'templates']` 경로 추가 (2단원)
`NoReverseMatch at /...`	`{% url 'app:name' %}` 역참조 시 이름표 오타가 있거나 필수 인자(`article.pk`)를 누락함	`urls.py`의 `app_name`·`name` 철자와 `<int:pk>` 파라미터 전달 여부 확인 (3단원)
`DoesNotExist` / `MultipleObjectsReturned`	단일 알맹이를 꺼내는 `get()` 호출 시 결과가 0개이거나 2개 이상이어서 서버 에러(500) 폭발	`get()`은 오직 고유 번호(`pk`) 조회에만 쓰고, 실무에선 `get_object_or_404` 활용 (6단원)
`403 Forbidden (CSRF verification failed)`	상태를 바꾸는 `POST` 요청 시 정식 폼임을 증명하는 위조 방지 암구호 스탬프가 누락됨	`<form method="POST">` 바로 아랫줄에 `{% csrf_token %}` 태그 삽입 (7단원)
`ValueError: didn't return an HttpResponse`	통합 `create`/`update` 함수에서 마지막 `return render()`를 `else:` 안으로 잘못 들여씀	유효성 검사(`is_valid()`) 실패 시에도 도달하도록 맨 아래 `return render()`를 `if-else` 바깥에 배치 (8단원)

실전 상황 / 에러 메시지	왜 이런 현상이 발생했을까? (근본 원리)	베테랑의 10초 해결책 & 관련 단원
회원정보 수정 창에 관리자 권한 항목이 다 뜸	내장 `UserChangeForm`은 관리자용(`fields='__all__'`)으로 설계되어 모든 필드를 노출함	`CustomUserChangeForm`의 `Meta`에서 `fields = ('first_name', 'last_name', 'email')`로 제한 (10단원)
비밀번호를 바꿨더니 갑자기 로그아웃됨	비밀번호가 바뀌면 DB 해시값이 달라져 기존 세션의 인증 해시와 불일치하여 무효화됨	`user = form.save()` 바로 아랫줄에 `update_session_auth_hash(request, user)` 호출 (9·10단원)
비밀번호 초기화 시 `ConnectionRefusedError`	실제 이메일 발송(SMTP) 서버가 설정되지 않은 상태에서 메일 전송을 시도함	`settings.py`에 `EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'` 추가해 터미널에서 링크 확인 (10단원)
`SystemCheckError: Reverse accessor 'Group.user_set' clashes (fields.E304)`	`accounts/models.py`에 `User` 클래스만 만들고 `settings.py`에 `AUTH_USER_MODEL` 등록을 빼먹어 두 User 모델이 충돌함	`settings.py`에 `AUTH_USER_MODEL = 'accounts.User'`를 추가해 기본 User를 완전히 교체 (11단원)
`AttributeError: Manager isn't available; 'auth.User' has been swapped`	커스텀 User로 교체해 놓고 회원가입 시 기본 `UserCreationForm`을 상속·교체 없이 그대로 사용함	`CustomUserCreationForm(UserCreationForm)`에서 `Meta.model = get_user_model()`로 교체 후 사용 (9·11단원)